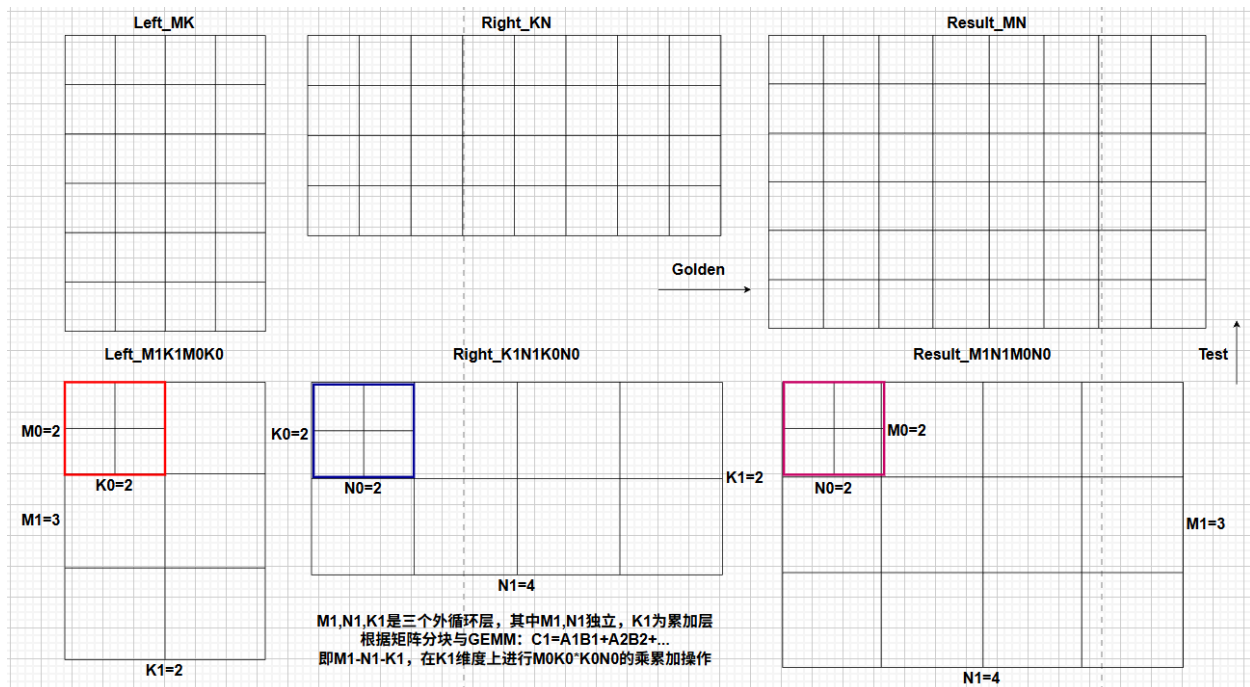
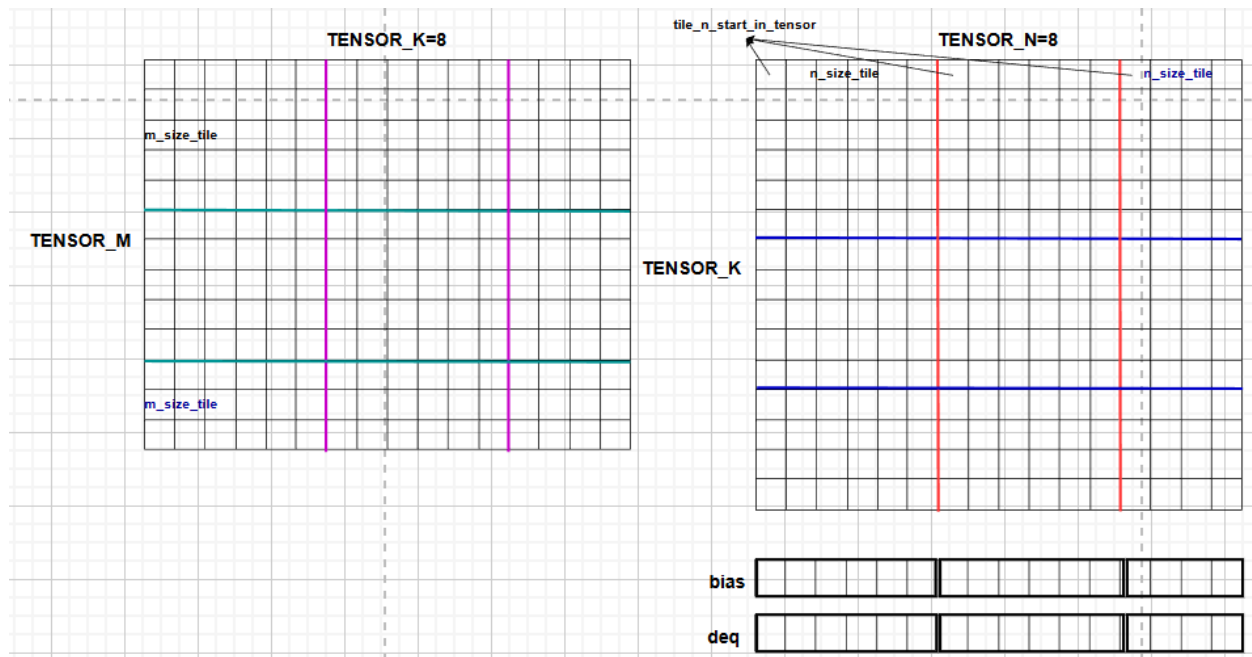


python模拟硬件行为

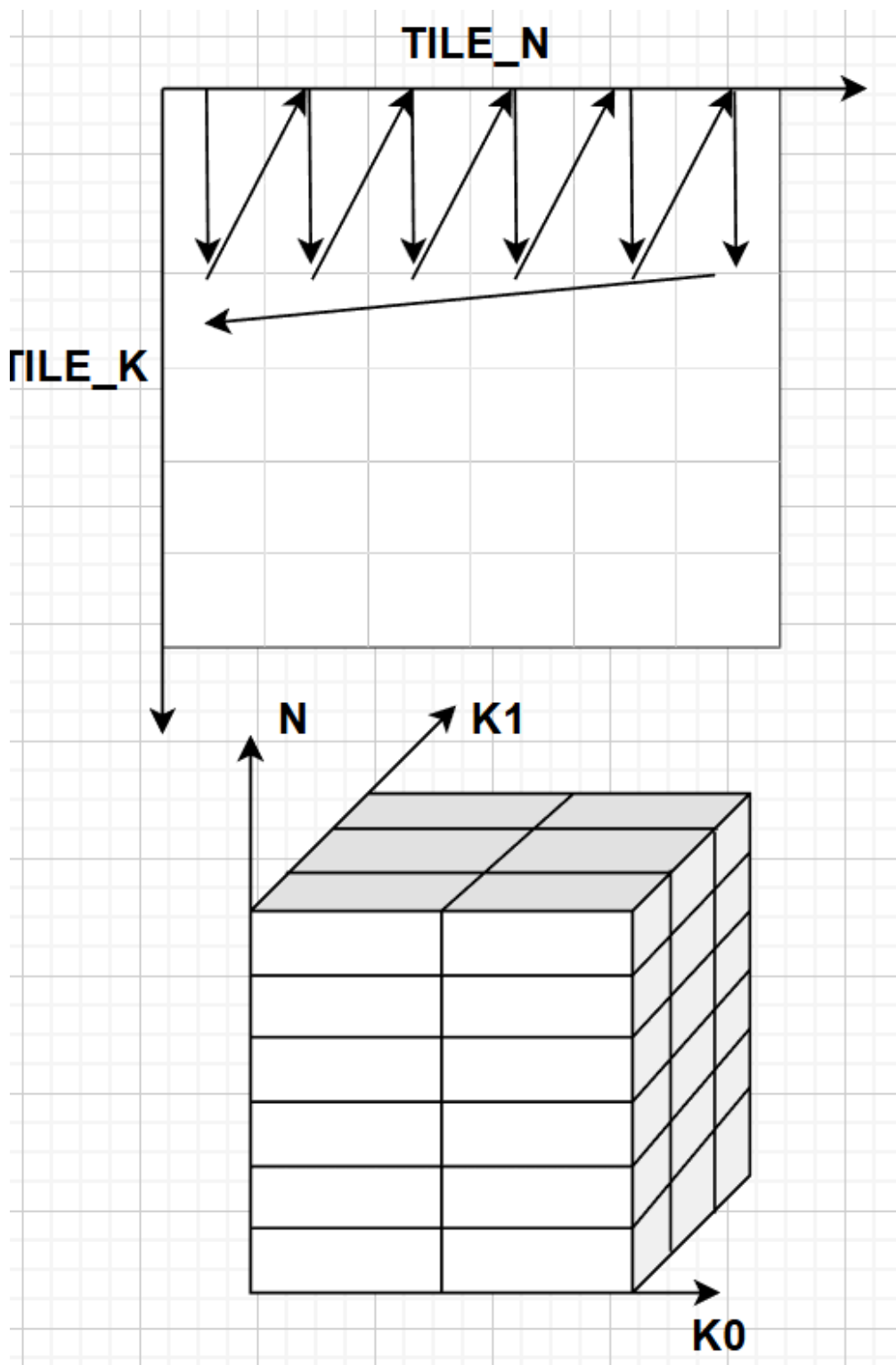
Step1.py：输入矩阵的普通GEMM



Step2.py：输入矩阵，且添加三级缓存、bias、deq的GEMM



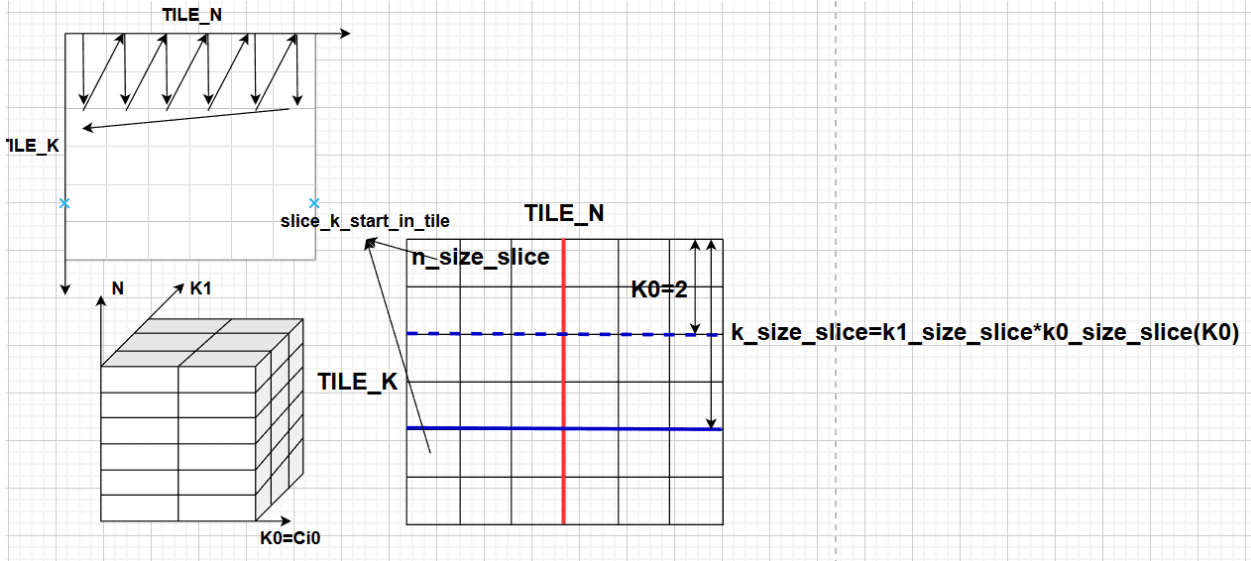
result_m2n2m1n1m0n0_psb : 是tile层的



K 轴拆分时，*numpy*已经做了自动填充，需要 $k_size_tile_align_k0$ 对齐填充后数据

$MK2K1MK0_DDR2L1$ 采用" $tensor$ 维度的固定值和 $tile$ 维度的偏移量" 进行拆轴和移轴，最终从 $L2$ 的 $MK \rightarrow L1$ 的 $K1MK0$

参考 $Tensor \rightarrow Tile$ 可以做出 $Tile \rightarrow Slice$ ，采用" $tile$ 维度的固定值和 $slice$ 维度的偏移量"，在这个过程中 $L1$ 的 $K1MK0 \rightarrow L0$ 的 $M1K1M0K0$



- 因为每 $K0$ 个连续的 K 构成一个 $K1$ 块，则有：
- $k1_size_slice = k_size_slice // K0$
- $slice_k1_start_in_tile = slice_k_start_in_tile // K0$

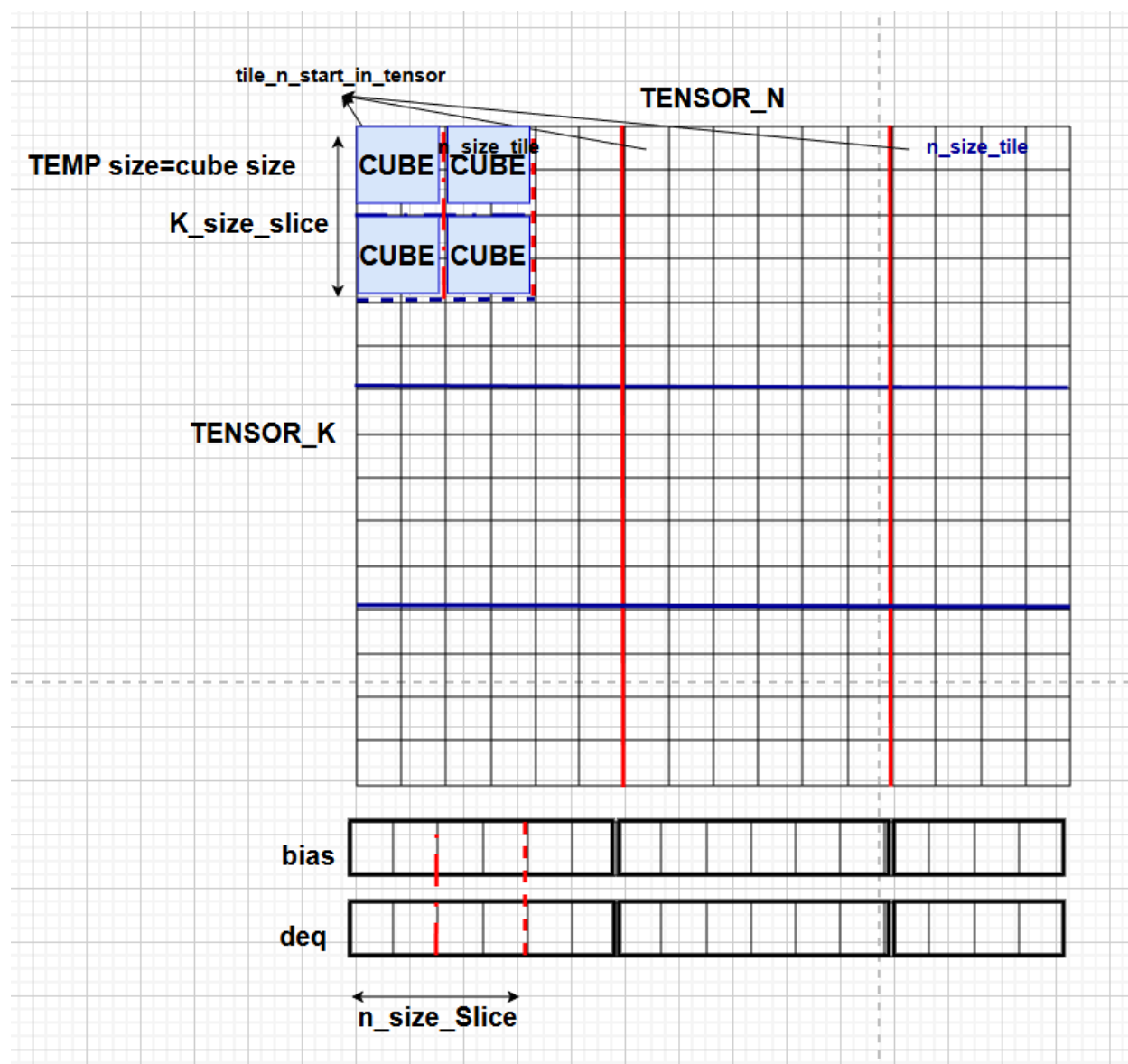
由于 K 维是累加维，且最后 $matmulmatmul$ 算是在 $slice$ 层执行，运算细粒度为 $Cube : M0/N0/K0$ ，内部遍历一个 $slice$ 内的所有 $cube$

$matmul$ 接收一个 $slice$ 的数据，内部按 $cube (m1, n1, k1)$ 遍历：

- **$k1$ 循环**：同一个 $(m1, n1)$ 位置的所有 $Kcube$ 做乘加累加到 `temp`
- **$bias$** ：对当前 $(m1, n1)$ 的 `temp` 加偏置
- **$psum$** ：决定 `temp` 是覆盖还是累加到 PSB 的 `m1n1m0n0[m1][n1]`
- **deq** ：对 PSB 的 `m1n1m0n0[m1][n1]` 做反量化

每个 $(m1, n1)$ 独立操作各自的 $(M0, N0)$ $cube$ 块，但 $bias/psum/deq$ 的使能信号是共享的——因为它们都作用在 $Kslice$ 位置上。

那么 $bias_en$ 是全局第一个 K_slice 切片， deq 是最后一个 K_slice 切片， $psum_en$ 为 $not\ bias_en$



Step3.py：输入特征图ifm转换成矩阵layout (im2col)

1. $K_{h/w\ eff} = Dilation \times (K_{h/w} - 1) + 1$
2. $H/W_o = \frac{H/W_i + padding \times 2 - K_{heff}}{stride} + 1$
3. 根据输入特征图的 $H_i W_i C_i$ 和 $Co C_i K_h K_w$ 计算 $Ho Wo$
4. 遍历 Ho, Wo, K_h, K_w ，计算 ho, wo 对应的 hi, wi

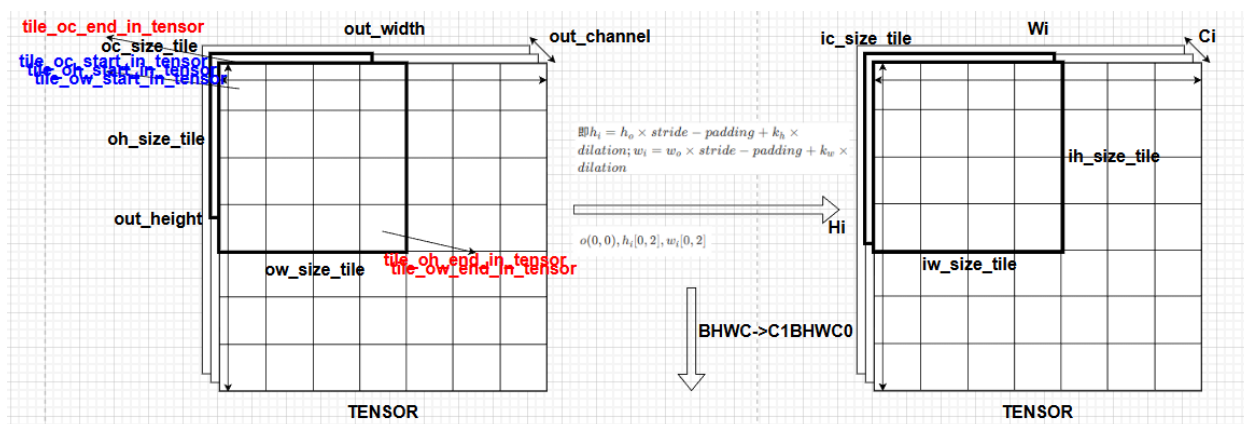
5. 遍历 C_i ，则可以进行mapping，按照所需生长顺序转换成 MK 矩阵：

6. $M = HoWo \rightarrow m = ho * Wo + wo$

7. $K = CiKhKw \rightarrow k = ci * Kh * Kw + kh * Kw + kw$

8. $ofm[m][k] = ifm[ci][hi][wi]$

Step4.py：输入特征图ifm，加入三级缓存，bias，deq，转换为GEMM运算的全流程模拟



1. 计算预期的 ofm 大小，进行 ofm tensor的tile切分，得到 $oh/w/c_size_tile$

2. 根据 $start/end_oh/w_tile_in_tensor$ 计算 ih/w 坐标，得到 ih/w_size_tile

3. $BHWC \rightarrow C1BHWCO$ 完成 $L2 \rightarrow L1$

4. 计算预期的 ofm tile的slice切分，得到 $oh/w/c_size_slice$

5. 根据 $start/end_oh/w_slice_in_tile$ 计算 ih/w 坐标，得到 ih/w_size_slice

6. 在slice层完成卷积padding，根据输入参数 pad_h/w 和边界条件得到 $pad_t/b/l/r$ 的value

a. $slice_ih/w_start/end_in_tile$ 做了 max/min 的边界处理，这里需要使用 ofm 反推

```
if(slice_oh_start_in_tensor * stride_h - pad_h < 0):
    slice_pad_t = pad_h - slice_oh_start_in_tensor * stride_h
```

```

else:
    slice_pad_t = 0
if(slice_oh_end_in_tensor * stride_h - pad_h + dilation_h
    * (kernel_h - 1) > in_height - 1):
    slice_pad_b = slice_oh_end_in_tensor * stride_h - pad_h
    + dilation_h * (kernel_h - 1) - (in_height - 1)
else:
    slice_pad_b = 0
if(slice_ow_start_in_tensor * stride_w - pad_w < 0):
    slice_pad_l = pad_w - slice_ow_start_in_tensor * stride_w
else:
    slice_pad_l = 0
if(slice_ow_end_in_tensor * stride_w - pad_w + dilation_w
    * (kernel_w - 1) > in_width - 1):
    slice_pad_r = slice_ow_end_in_tensor * stride_w - pad_w
    + dilation_w * (kernel_w - 1) - (in_width - 1)
else:
    slice_pad_r = 0

```

i. 注意：顶层 $Kh/w = 0$ ，底层为 $Size - 1$

ii. 注意： $end - start + 1 = size$

7. 在输入是 $matrix$ 时， $bias_en$ 是全局第一个 K_slice 切片， deq 是最后一个 K_slice 切片， $psum_en$ 为 $not\ bias_en$ ；那么输入是 $BHWC$ 时，累加维度是 Ci ，所以 $bias/deq_en$ 是全局第一个/最后一个 Ci 切片，由于 $C0 = K0$ ，且在 $L2 \rightarrow L1$ 的过程中已经做了 $C1C0$ 拆轴移轴，故全局第一个/最后一个 Ci 切片等效于下式， $psum_en$ 依然是 $not\ bias_en$

```

bias_en = (tile_ic_start_in_tensor==0) and (slice_cil_start_in_tile==0)
deq_en = (tile_ic_start_in_tensor + ic_size_tile >= in_channel) and (slice_cil_start_in_tile + slice_cil_size >= cil_size_tile)

```

8. 进行 $L1 \rightarrow L0$, 将 $C1BHW C0$ 转换成 $M1K1M0K0$ 的矩阵格式, 根据下式转换, 注意 $C0 = K0$ 已经固定, 只需使用 $ic1 \rightarrow k1_idx$ 即可

```
m_index = b*slice_oh*slice_ow + oh*slice_ow + ow
kl_idx = icl * kernel h * kernel w + kh * kernel w + kw
```

Q: 这段好像有点问题，我认为应该CO_L0 → oc_size_tile

```
# im2col操作后特征图使用矩阵表达的维度:  $M=H_o \times W_o$ ,  $K=C_i K_h K_w$ ,  $N=C_o$ 
# 倒推做im2col膨胀前使用矩阵表达的维度:  $M=H_i \times W_i$ ,  $K=C_i$ ,  $N=C_o$ 
# 矩阵在L1的layout是 $K1MK0$ , 特征图在L1的layout是 $Ci1HiWiCi0$ 
M = batch * H_L0 * W_L0
N = oc_size_tile
```

PS

卷积 padding :

- 阶段： $L1 \rightarrow L0$ (im2col 时)
- 原因：L1 没存 padding 的 0，但卷积语义要求边界补 0
- 体现：`slice_pad_t/b/l/r`，动态反映全局 `pad_h/pad_w`
- 补的是**有意义的 0**，参与计算，影响最终结果

对齐 padding：(python代码在L2→L1中numpy拆轴时隐式做了K维度的对齐，硬件中应该要避免，因为前文提过L1可以不是Cube整数倍)

- 阶段：L2 $\rightarrow$ L1 (K 轴拆成 $K1 \times K0$ 时补 0) 和 L1 $\rightarrow$ L0 (M/N 轴拆成 $M1 \times M0$, $N1 \times N0$ 时越界位置为 0)
- 原因：硬件一次算 $M0 \times K0 \times N0$, shape 必须是整数倍
- 体现：`np.zeros` 初始化 + 越界跳过
- 补的是无意义的 0, 不影响最终结果 (乘 0 不贡献累加值, 或写回时不超出有效范围)

L1→L0的padding操作

- 可以分成两个padding操作，第一个操作是全局padding，这个是标记，L2/L1没有任何无效数据0，在L1-L0中，全局padding使用slice_pad_b/t/l/r进行恢复，第二个是对齐padding，因为给MXU的必须是cube，L0存的就必须是cube整数倍，所以L1-L0还要做一步对齐

1. 卷积 padding 恢复：

- 全局 `pad_h/pad_w` 只是标记，L2/L1 不存任何 padding 的 0
- L1→L0 时根据 `slice_pad_t/b/l/r` 在 receptive field 边界补 0
- 目的：还原卷积语义

2. 对齐 padding：

- L0 送给 MXU (Cube Unit) 的必须是 `(M1, K1, M0, K0)` 格式
- 如果 slice_m 不是 M0 整数倍、slice_k 不是 K0 整数倍，需要补 0 凑齐
- 目的：满足硬件计算粒度

C1BHWCO → M1K1M0K0

$$C_0 = K_0$$

$$BHW = BH_oW_o * C_iK_hK_w = MK1 = M1K1M0$$

$$M1M0 = BH_oW_o$$

$$K_1 = C_iK_hK_w$$